

Khmelnyskyi National University
Department of Information Technonogy

Report

Laboratory work №4

Discipline: “Programming”

Topic: “*Arrays. Work with one-dimensional arrays*”

Completed: 1st year student, group SE-26-1

Nykon Hrytsyshyn
Name, Surname

Checked:

Valeriy Martynyuk
Name, Surname

Purpose: Study of basic algorithms for processing one-dimensional arrays.

Technical Environment: The project is implemented in accordance with the C11 standard and configured for cross-platform compilation using CMake and the Ninja build system. The application is built and verified using modern C compilers (GCC, Clang, and MSVC) with strict diagnostic flags enabled, ensuring high code quality, portability, and compatibility across operating systems.

Note: The software implementation is designed using a modular architecture with robust input validation and an interactive task execution engine. Detailed flowcharts illustrating the overall execution logic, along with full source code for all program components (tasks.h, utils.h, main.c, runner.c, and utils.c), are provided in Appendices A...F.

Task 1

A sequence of N numbers is given: $a(0), a(1), a(2), \dots, a(N)$.

Find:

- the largest number with an even number;
- the smallest number with an odd number;
- the closest number to the arithmetic mean of these numbers;
- the number of even numbers.

The algorithm processes a one-dimensional array a of N integers ($1 \leq N \leq 100$) to evaluate extreme elements at even/odd indices, compute the arithmetic mean $\bar{a}$, identify the array element closest to $\bar{a}$, and count total even element values.

The exact arithmetic mean is computed as $\bar{a} = \frac{\text{sum}}{N}$. A secondary linear scan iterates i from 1 to $N - 1$, comparing absolute deviations $|a[i] - \bar{a}|$ via fabs() to identify closest_index minimizing distance to the mean. The array is formatted using print_sequence(), followed by outputting the computed extremums, mean, nearest element, and parity statistics. Edge cases where no odd indices exist ($N = 1$) are explicitly handled during display.

					LW-SE-26-1 09		
Ch.	Sheet	N° document	Signature	Date			
Developed		Hrytsyshyn N.B.					
Checked		Martynyuk V.V.					
N.control							
Confirmed							
Laboratory work N°4					Letter	Sheet	Sheets
					H	1	21
					KNU group SE-26-1		

1.1. Block diagram of the program algorithm

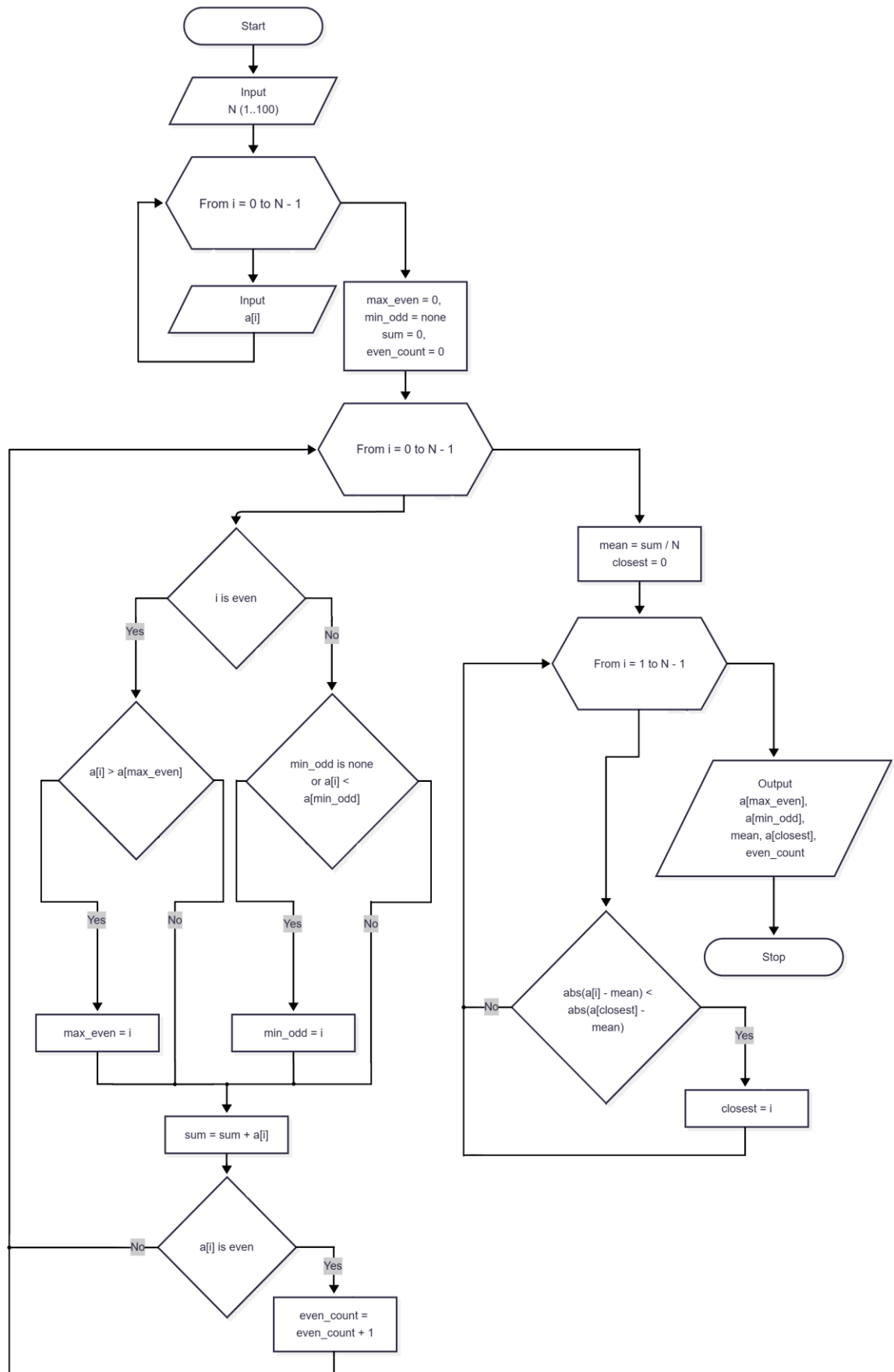


Fig. 1 – Block diagram of the program algorithm

1.2. Program text

```
#include "tasks.h"
#include "utils.h"
#include <math.h>
#include <stdio.h>

void run_task1(void) {
    long a[MAX_ARRAY_SIZE];
    long n = 0;

    puts("\n=== Task 1: extreme elements, mean and even numbers ===\n");

    if (!read_sequence("a", a, &n)) {
        return;
    }

    long max_even_index = 0;
    long min_odd_index = -1;
    long sum = 0;
    long even_count = 0;

    for (long i = 0; i < n; ++i) {
        if (i % 2 == 0) {
            if (a[i] > a[max_even_index]) {
                max_even_index = i;
            }
        } else if (min_odd_index < 0 || a[i] < a[min_odd_index]) {
            min_odd_index = i;
        }

        sum += a[i];

        if (a[i] % 2 == 0) {
            ++even_count;
        }
    }

    const double mean = (double)sum / (double)n;
    long closest_index = 0;

    for (long i = 1; i < n; ++i) {
        if (fabs((double)a[i] - mean) < fabs((double)a[closest_index] - mean)) {
            closest_index = i;
        }
    }
```

```

    }

    puts("");
    print_sequence("a", a, n);
    printf(
        "\nLargest element with an even ordinal number: a[%ld] = %ld\n",
        max_even_index,
        a[max_even_index]
    );

    if (min_odd_index < 0) {
        puts("Smallest element with an odd ordinal number: none (N = 1)");
    } else {
        printf(
            "Smallest element with an odd ordinal number: a[%ld] = %ld\n",
            min_odd_index, a[min_odd_index]
        );
    }

    printf(
        "Arithmetic mean: %.4f, the closest element: a[%ld] = %ld\n",
        mean, closest_index, a[closest_index]
    );
    printf("Number of even elements: %ld\n", even_count);
}

```

1.3. Program results

```

=== Task 1: extreme elements, mean and even numbers ===

Enter the number of elements N: 7
a[0] = 4
a[1] = -3
a[2] = 10
a[3] = 8
a[4] = -3
a[5] = 6
a[6] = 5

a = [4, -3, 10, 8, -3, 6, 5]

Largest element with an even ordinal number: a[2] = 10
Smallest element with an odd ordinal number: a[1] = -3
Arithmetic mean: 3.8571, the closest element: a[0] = 4
Number of even elements: 4

```

Fig. 2 – Task 1 results

Task 2

Given a sequence of N numbers: $a(0), a(1), a(2), \dots, a(N)$. Find the sum of numbers that have even ordinal numbers and are even.

2.1. Block diagram of the program algorithm

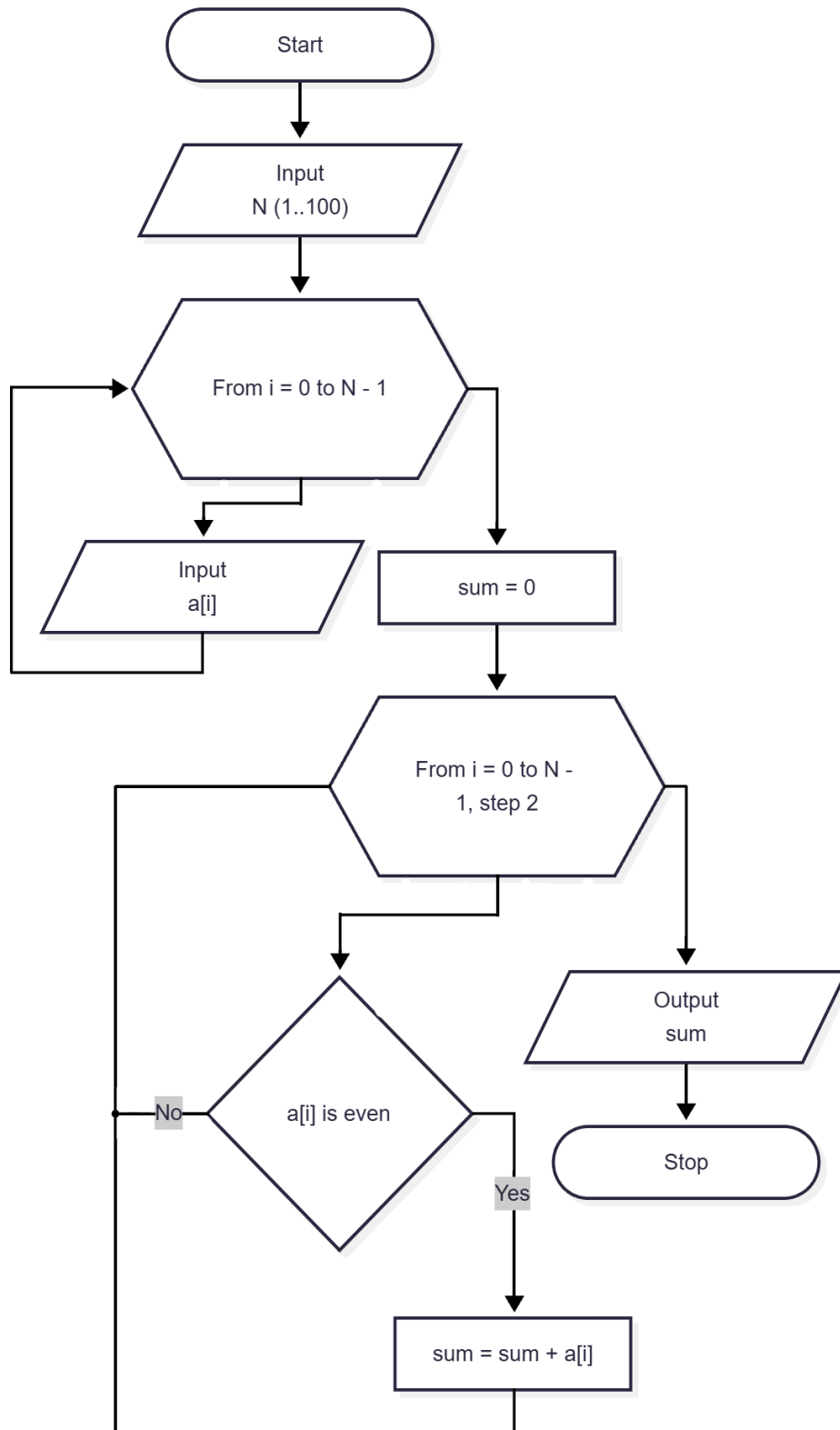


Fig. 3 – Block diagram of the program algorithm

2.2. Program text

```
#include "tasks.h"
#include "utils.h"

#include <stdio.h>

void run_task2(void) {
    long a[MAX_ARRAY_SIZE];
    long n = 0;

    puts("\n=== Task 2: sum of even elements with even ordinal numbers ===\n");

    if (!read_sequence("a", a, &n)) {
        return;
    }

    long sum = 0;

    for (long i = 0; i < n; i += 2) {
        if (a[i] % 2 == 0) {
            sum += a[i];
        }
    }

    puts("");
    print_sequence("a", a, n);
    printf("\nSum: %ld\n", sum);
}
```

2.3. Program results

```
=== Task 2: sum of even elements with even ordinal numbers ===

Enter the number of elements N: 6
a[0] = 2
a[1] = 4
a[2] = 6
a[3] = 8
a[4] = 3
a[5] = 10

a = [2, 4, 6, 8, 3, 10]

Sum: 8
```

Fig. 4 – Task 2 results

Task 3

Given a sequence of N numbers: $a(0), a(1), a(2), \dots, a(N)$. Find the sum of numbers that have even ordinal numbers and are even.

3.1. Block diagram of the program algorithm

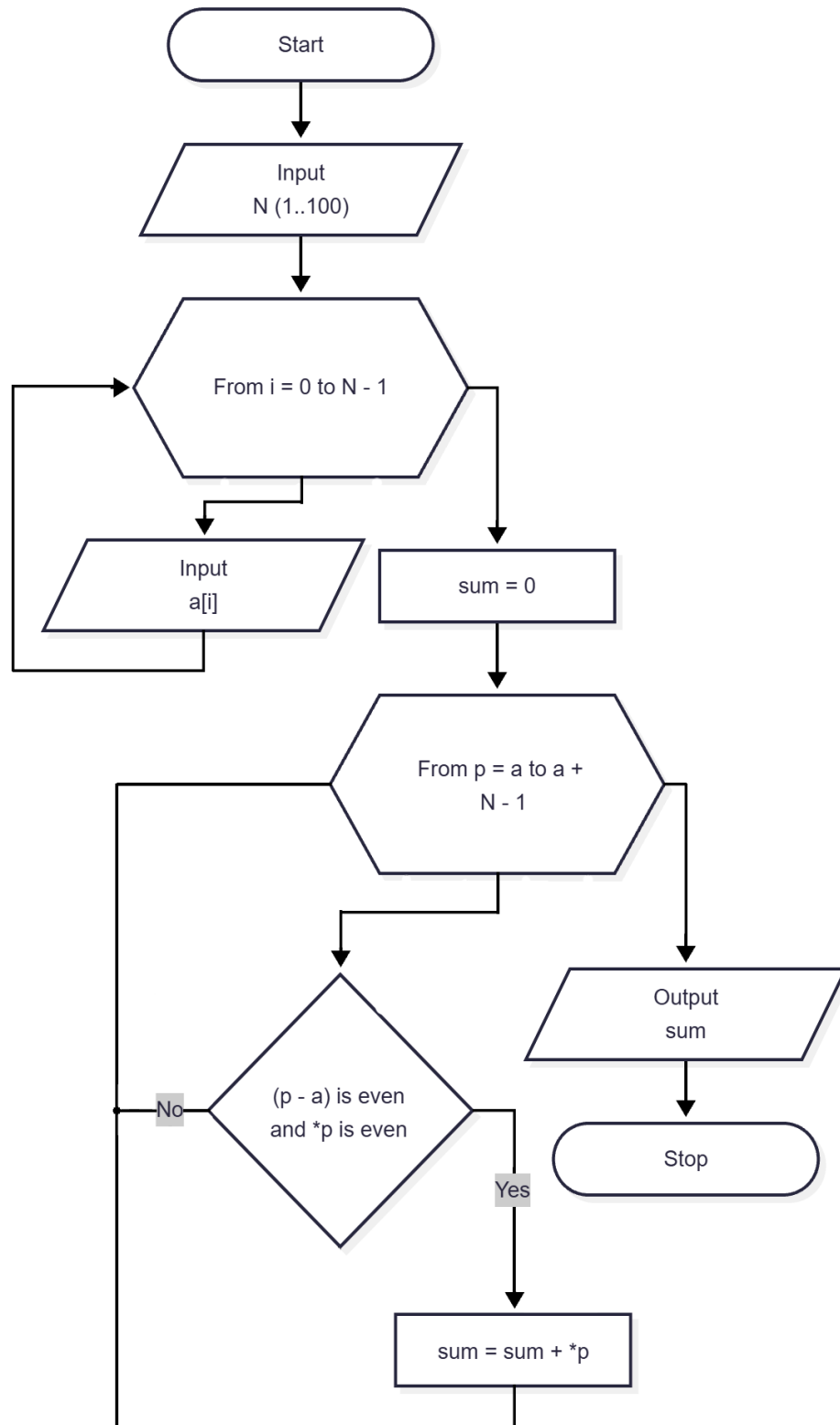


Fig. 5 – Block diagram of the program algorithm

3.2. Program text

```
#include "tasks.h"
#include "utils.h"
#include <stdio.h>

void run_task3(void) {
    long a[MAX_ARRAY_SIZE];
    long n = 0;

    puts(
        "\n=== Task 3: sum of even elements with even ordinal numbers (through "
        "a pointer) ==="
    );

    if (!read_sequence("a", a, &n)) {
        return;
    }

    long sum = 0;

    for (const long *p = a; p < a + n; ++p) {
        const ptrdiff_t ordinal = p - a;

        if (ordinal % 2 == 0 && *p % 2 == 0) {
            sum += *p;
        }
    }

    puts("");
    print_sequence("a", a, n);
    printf("\nSum: %ld\n", sum);
}
```

3.3. Program results

```
=== Task 3: sum of even elements with even ordinal numbers (through a pointer) ===
Enter the number of elements N: 5
a[0] = 2
a[1] = 4
a[2] = 6
a[3] = 8
a[4] = 3

a = [2, 4, 6, 8, 3]

Sum: 8
```

Fig. 6 – Task 3 results

Conclusions

During the execution of this laboratory work, practical skills were acquired in processing one-dimensional arrays in C11, implementing fundamental array manipulation algorithms, and utilizing pointer arithmetic for array traversal and element addressing.

APPENDIX A

Block diagram of the main program control flow algorithm

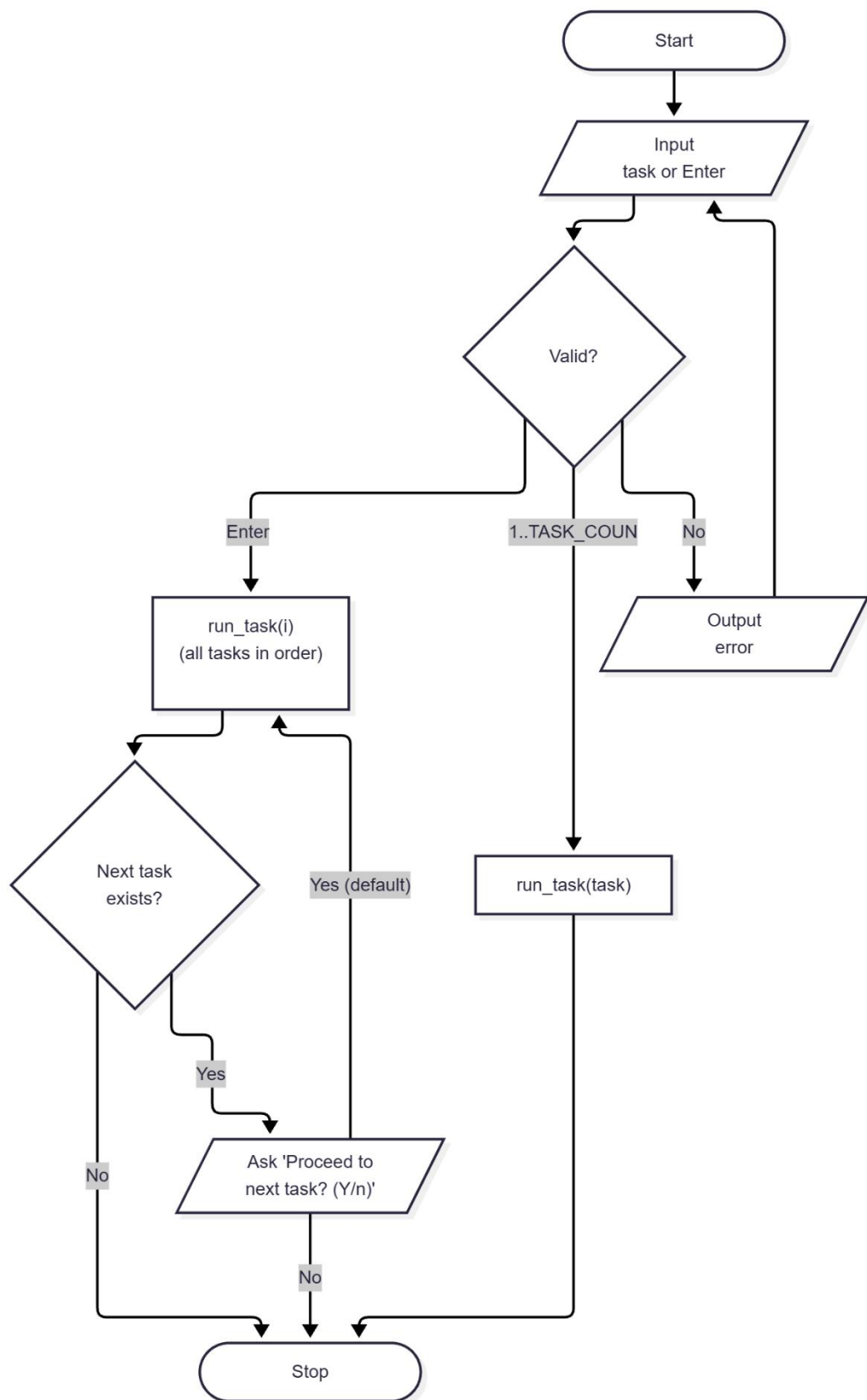


Fig. A.1 – Block diagram of the main program control flow algorithm

APPENDIX B

Source code of the “tasks.h” file

```
#ifndef TASKS_H
#define TASKS_H

#define TASK_LIST X(1) X(2) X(3)

#define X(num) void run_task##num(void);
TASK_LIST
#undef X

#define X(num) +1
enum { TASK_COUNT = 0 TASK_LIST };
#undef X

void run_task(int n);
void run_all_tasks(void);

#endif // TASKS_H
```

APPENDIX C

Source code of the “utils.h” file

```
#ifndef UTILS_H
#define UTILS_H

#include <stdbool.h>
#include <stddef.h>

#define MAX_ARRAY_SIZE 100
#define MAX_ELEMENT_ABS 1000000L

bool read_double(const char *prompt, double *out_value);
bool read_doubles(const char *prompt, size_t count, ...);
bool read_long(const char *prompt, long *out_value);
bool read_longs(const char *prompt, size_t count, ...);

bool read_long_in_range(
    const char *prompt,
    long min,
    long max,
    long *out_value
);
bool read_optional_long_in_range(
    const char *prompt,
    long min,
    long max,
    long *out_value,
    bool *out_provided
);

bool read_sequence(const char *name, long *array, long *out_count);
void print_sequence(const char *name, const long *array, long count);

bool confirm(const char *prompt, bool default_answer);

#endif // UTILS_H
```

APPENDIX D

Source code of the “main.c” file

```
#include "tasks.h"
#include "utils.h"
#include <stdbool.h>
#include <stdio.h>

#define PROMPT_SIZE 96

int main(void) {
    puts(
        "=====\\n"
        "  Laboratory work 4: One-dimensional arrays (variant 9)\\n"
        "=====\\n"
    );

    char prompt[PROMPT_SIZE];
    snprintf(
        prompt,
        sizeof(prompt),
        "Enter task number (1-%d) or press Enter to run all tasks [ALL]: ",
        TASK_COUNT
    );

    long task = 0;
    bool provided = false;

    if (!read_optional_long_in_range(prompt, 1, TASK_COUNT, &task, &provided))
    {
        return 1;
    }

    if (provided) {
        run_task((int)task);
    } else {
        run_all_tasks();
    }

    printf("\\nPress Enter to exit...");
    getchar();

    return 0;
}
```

APPENDIX E

Source code of the “runner.c” file

```
#include "tasks.h"
#include "utils.h"

#include <stdio.h>

static void (*const TASKS[TASK_COUNT])(void) = {
#define X(num) run_task##num,
    TASK_LIST
#undef X
};

void run_task(const int n) {
    TASKS[n - 1]();
}

void run_all_tasks(void) {
    for (int i = 1; i <= TASK_COUNT; ++i) {
        run_task(i);

        if (i < TASK_COUNT) {
            char prompt[64];
            snprintf(
                prompt, sizeof(prompt), "\nProceed to task %d? (Y/n): ", i + 1
            );

            if (!confirm(prompt, true)) {
                break;
            }
        }
    }
}
```

APPENDIX F

Source code of the “utils.c” file

```
#include "utils.h"

#include <ctype.h>
#include <errno.h>
#include <stdarg.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define PROMPT_SIZE 32

enum { MAX_READ_VALUES = 8 };

typedef bool (*token_parser_fn)(const char *str, char **endptr, void *out);

static bool is_buffer_empty(const char *ptr) {
    while (*ptr != '\0') {
        if (!isspace((unsigned char)*ptr)) {
            return false;
        }
        ptr++;
    }

    return true;
}

static bool parse_double_token(const char *str, char **endptr, void *out) {
    errno = 0;
    *(double *)out = strtod(str, endptr);

    return *endptr != str && errno == 0;
}

static bool parse_long_token(const char *str, char **endptr, void *out) {
    errno = 0;
    *(long *)out = strtol(str, endptr, 10);

    return *endptr != str && errno == 0;
}

static bool read_line_values(
```

```

const char *prompt,
const size_t count,
void *const outs[],
const token_parser_fn parse
) {
    if (count == 0 || count > MAX_READ_VALUES) {
        return false;
    }

    if (prompt != NULL) {
        printf("%s", prompt);
    }

    char buffer[256];
    if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
        return false;
    }

    buffer[strcspn(buffer, "\r\n")] = '\0';

    const char *ptr = buffer;

    for (size_t i = 0; i < count; ++i) {
        if (outs[i] == NULL) {
            return false;
        }

        char *endptr;
        if (!parse(ptr, &endptr, outs[i])) {
            return false;
        }

        ptr = endptr;
    }

    return is_buffer_empty(ptr);
}

bool read_double(const char *prompt, double *out_value) {
    return read_doubles(prompt, 1, out_value);
}

bool read_doubles(const char *prompt, const size_t count, ...) {
    if (count == 0 || count > MAX_READ_VALUES) {
        return false;
    }

```

```

    }

    void *outs[MAX_READ_VALUES];

    va_list args;
    va_start(args, count);

    for (size_t i = 0; i < count; ++i) {
        outs[i] = va_arg(args, double *);
    }

    va_end(args);

    return read_line_values(prompt, count, outs, parse_double_token);
}

bool read_long(const char *prompt, long *out_value) {
    return read_longs(prompt, 1, out_value);
}

bool read_longs(const char *prompt, const size_t count, ...) {
    if (count == 0 || count > MAX_READ_VALUES) {
        return false;
    }

    void *outs[MAX_READ_VALUES];

    va_list args;
    va_start(args, count);

    for (size_t i = 0; i < count; ++i) {
        outs[i] = va_arg(args, long *);
    }

    va_end(args);

    return read_line_values(prompt, count, outs, parse_long_token);
}

bool read_long_in_range(
    const char *prompt,
    const long min,
    const long max,
    long *out_value
) {

```

```

while (true) {
    if (read_long(prompt, out_value)) {
        if (*out_value >= min && *out_value <= max) {
            return true;
        }

        printf("Error: value must be between %ld and %ld.\n", min, max);
    } else if (feof(stdin)) {
        return false;
    } else {
        puts("Invalid input. Please enter an integer.");
    }
}
}

```

```

bool read_optional_long_in_range(
    const char *prompt,
    const long min,
    const long max,
    long *out_value,
    bool *out_provided
) {
    char buffer[256];

    while (true) {
        printf("%s", prompt);

        if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
            return false;
        }

        if (is_buffer_empty(buffer)) {
            *out_provided = false;
            return true;
        }

        char *endptr;
        long value;

        if (!parse_long_token(buffer, &endptr, &value) ||
            !is_buffer_empty(endptr)) {
            puts("Invalid input. Please enter an integer or press Enter.");
        } else if (value < min || value > max) {
            printf("Error: value must be between %ld and %ld.\n", min, max);
        } else {

```

```

        *out_value = value;
        *out_provided = true;
        return true;
    }
}

bool read_sequence(const char *const name, long *const array, long *out_count) {
    long count = 0;

    if (!read_long_in_range(
        "Enter the number of elements N: ", 1, MAX_ARRAY_SIZE, &count
    )) {
        return false;
    }

    char prompt[PROMPT_SIZE];

    for (long i = 0; i < count; ++i) {
        snprintf(prompt, sizeof(prompt), "%s[%ld] = ", name, i);

        if (!read_long_in_range(
            prompt, -MAX_ELEMENT_ABS, MAX_ELEMENT_ABS, &array[i]
        )) {
            return false;
        }
    }

    *out_count = count;

    return true;
}

void print_sequence(
    const char *const name,
    const long *const array,
    const long count
) {
    printf("%s = [", name);

    for (long i = 0; i < count; ++i) {
        printf(i == 0 ? "%ld" : ", %ld", array[i]);
    }

    puts("]");
}

```

```

}

static bool equals_ignore_case(
    const char *str,
    const size_t len,
    const char *word
) {
    if (strlen(word) != len) {
        return false;
    }

    for (size_t i = 0; i < len; ++i) {
        if (tolower((unsigned char)str[i]) != word[i]) {
            return false;
        }
    }

    return true;
}

bool confirm(const char *prompt, const bool default_answer) {
    char buffer[16];

    while (true) {
        if (prompt != NULL) {
            printf("%s", prompt);
        }

        if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
            return false;
        }

        const bool overflow = strchr(buffer, '\n') == NULL && !feof(stdin);
        if (overflow) {
            int ch;
            while ((ch = getchar()) != '\n' && ch != EOF)
                ;
        } else {
            const char *start = buffer;
            while (isspace((unsigned char)*start)) {
                start++;
            }

            size_t len = strlen(start);
            while (len > 0 && isspace((unsigned char)start[len - 1])) {

```

```

        len--;
    }

    if (len == 0) {
        return default_answer;
    }

    if (equals_ignore_case(start, len, "y") ||
        equals_ignore_case(start, len, "yes")) {
        return true;
    }

    if (equals_ignore_case(start, len, "n") ||
        equals_ignore_case(start, len, "no")) {
        return false;
    }
}

printf("Please answer 'y' or 'n'.\n");
}
}

```